

PROYECTO SEMESTRAL — ENTREGA 1

**Asignación de vueltas a buses de una misma ruta con
ventanas de inicio y tiempos de viaje dependientes del
tráfico**

*Heurísticas List Scheduling y LPT para máquinas idénticas, con Ramificación y Poda y
CP-SAT como referencia exacta, aplicadas al transporte urbano de Cusco*

Asignatura	Algoritmos Avanzados
Docente	Héctor Eduardo Ugarte Rojas
Grupo	Grupo 4
Tema asignado	Optimización de asignación de trabajos en máquinas idénticas
Atributo del graduado	AG_C06 – Aprendizaje a lo largo de la vida
Hito	Entrega 1 – Cierre del primer periodo

Integrantes	Código
Aguilar Quispe, Yon	111276
Choque Mamani, Brayan Isau	215719
Huacani de la Cruz, Dany	081561
Moreano Villena, Miguel Angel	211859

Resumen

Este trabajo aborda la **asignación de vueltas a los buses de una misma ruta** del transporte urbano de Cusco, modelada como un problema de programación de trabajos en máquinas paralelas idénticas: cada bus es una máquina y cada vuelta origen–destino–origen es un trabajo. El objetivo es asignar cada vuelta a un bus de su propia flota y fijar su hora de salida, de modo que se atienda la mayor cantidad de vueltas y, entre esas soluciones, se minimice la carga máxima de conducción por bus ($L_{\max}$). El modelo respeta ventanas de inicio, una jornada de 06:00 a 22:00 y un almuerzo de dos horas por bus.

El tráfico se incorpora haciendo que la duración de cada vuelta, $p_i(s_i)$, dependa de su hora de salida s_i . Para ello se usa un perfil de congestión por franjas horarias con velocidad constante por tramos, que garantiza que salir más tarde nunca implica llegar antes (propiedad FIFO). Se evalúan tres escenarios experimentales (sin tráfico, perfil normalizado y perfil pesimista) y, mediante simulación Monte Carlo, la variabilidad aleatoria de los tiempos de viaje.

Se implementaron en Python las heurísticas **List Scheduling (LS)** y **LPT** adaptadas a estas restricciones, junto con dos métodos exactos de referencia para instancias pequeñas (Ramificación y Poda y un modelo en OR-Tools CP-SAT), un validador independiente y 35 pruebas automatizadas. La experimentación usa las 37 empresas/rutas de referencia (5 017 vueltas) e instancias pequeñas generadas con semilla fija.

Los resultados preliminares muestran que, sin tráfico, LS y LPT producen la misma solución. Con tráfico, LS es superior: en el escenario normalizado deja 167 vueltas sin atender frente a 331 de LPT, con una brecha media de $L_{\max}$ respecto de la cota inferior de 13.7 % frente a 19.9 %. En instancias pequeñas, LS queda en promedio a 8–10 % del óptimo. Se concluye que, cuando las vueltas tienen ventanas de inicio y duraciones dependientes de la hora, procesarlas en orden temporal (LS) es más adecuado que priorizar las más largas (LPT), y que la hora de salida es una decisión con efecto propio sobre la carga.

Palabras clave: máquinas paralelas idénticas, List Scheduling, LPT, tiempos de viaje dependientes del tiempo, ventanas de inicio, ramificación y poda, CP-SAT, transporte urbano.

Índice

Resumen.....	1
1. Introducción.....	3
1.1 Contexto.....	3
1.2 Problema.....	3
1.3 Importancia de los tiempos de viaje dependientes de la hora.....	3
1.4 Objetivos.....	3
1.5 Metodología general.....	4
1.6 Alcance y delimitación.....	4
1.7 Organización del documento.....	4
2. Fundamentos y aprendizaje autónomo.....	5
2.1 Conceptos y algoritmos.....	5
2.2 Fuentes y criterio de selección.....	5
2.3 Conocimientos adicionales y aprendizaje autónomo.....	6
3. Stack tecnológico.....	7
4. Formulación del problema.....	8
4.1 Elementos del modelo.....	8
4.2 Origen de los datos.....	8

4.3 Modelo del tráfico.....	8
4.4 Entradas.....	9
4.5 Salidas.....	9
4.6 Restricciones.....	10
4.7 Supuestos.....	10
4.8 Función objetivo.....	10
4.9 Datos e instancias.....	11
5. Diseño algorítmico.....	12
5.1 Estrategias.....	12
5.2 Estructuras de datos.....	12
5.3 Pseudocódigo.....	13
5.4 Invariantes y corrección.....	14
5.5 Complejidad.....	14
6. Diseño del sistema.....	15
7. Ejemplo manual.....	16
7.1 Datos.....	16
7.2 Cálculo del tiempo de viaje con tráfico.....	16
7.3 List Scheduling.....	16
7.4 LPT.....	17
7.5 Resultados y comparación con la referencia exacta.....	18
8. Pruebas y validación.....	19
8.1 Validación de restricciones.....	19
8.2 Casos de prueba.....	19
8.3 Verificación de las salidas.....	19
9. Metodología experimental.....	21
9.1 Preguntas e hipótesis.....	21
9.2 Experimentos, instancias y variables.....	21
9.3 Entorno y procedimiento reproducible.....	21
10. Prototipo y resultados preliminares.....	22
10.1 Prototipo.....	22
10.2 E1 — Las 37 rutas.....	23
10.3 E2 — Comparación con el óptimo en instancias pequeñas.....	24
10.4 E3 — Escalabilidad.....	25
10.5 E4 — Robustez ante tráfico variable.....	25
10.6 Contraste con las hipótesis.....	26
11. Discusión y limitaciones.....	27
11.1 Interpretación.....	27
11.2 Limitaciones y dificultades pendientes.....	27
12. Conclusiones.....	28
13. Referencias.....	29
Anexo A. Manual de instalación y uso.....	30
Anexo B. Contribuciones y bitácora.....	31
Anexo C. Resultados por ruta.....	32

1. Introducción

1.1 Contexto

El transporte urbano de Cusco opera mediante empresas que atienden una ruta fija con su propia flota. Cada día la empresa decide qué bus realiza cada vuelta y a qué hora sale, respetando la frecuencia de servicio, el descanso del personal y una jornada limitada. Una mala asignación concentra horas de conducción en algunos buses mientras otros quedan ociosos, y puede dejar sin cubrir salidas en las horas de mayor demanda (Ibarra-Rojas et al., 2015).

1.2 Problema

Dada una empresa/ruta con M buses idénticos y un conjunto de vueltas con ventana de inicio, se busca **asignar cada vuelta a un bus de su propia ruta y fijar su hora de salida**. La asignación debe respetar la jornada de 06:00 a 22:00, el almuerzo de cada bus y la duración de cada vuelta según el tráfico. Se busca atender la mayor cantidad de vueltas y, entre esas soluciones, minimizar la carga de conducción del bus más cargado ($L_{\max}$).

Sin ventanas ni tráfico, el problema corresponde al clásico $P||C_{\max}$ de programación en máquinas paralelas idénticas, que es NP-difícil (Garey y Johnson, 1979). Las ventanas de inicio, el almuerzo y el tráfico lo hacen más restrictivo y justifican el uso de heurísticas evaluadas experimentalmente.

1.3 Importancia de los tiempos de viaje dependientes de la hora

Una vuelta que sale en hora punta tarda más que la misma vuelta en horario valle. Suponer una duración constante subestima la carga de los buses que trabajan en las puntas y sobreestima la de los que trabajan en el valle; además, impide valorar la hora de salida como decisión. Por eso el modelo representa la duración como una función $p_i(s_i)$ de la hora de salida, construida a partir de un perfil de congestión por franjas (Ichoua, Gendreau y Potvin, 2003).

1.4 Objetivos

Objetivo general

Desarrollar y evaluar experimentalmente una solución computacional basada en algoritmos de programación en máquinas idénticas para asignar las vueltas de las rutas de transporte urbano de Cusco, considerando ventanas de inicio, almuerzo y tiempos de viaje dependientes del tráfico.

Objetivos específicos

1. Formular el problema con entradas, salidas, restricciones, supuestos, función objetivo y un modelo explícito de tráfico.
2. Estudiar de forma autónoma los fundamentos de programación en máquinas paralelas, tiempos de viaje dependientes del tiempo y métodos exactos.
3. Implementar List Scheduling y LPT adaptados al problema, y un método exacto de referencia para instancias pequeñas.
4. Validar la corrección mediante un verificador de restricciones, una instancia resuelta a mano y casos de prueba básicos, límite, adversos y de escala.
5. Diseñar experimentos reproducibles que comparen las estrategias en factibilidad, $L_{\max}$, equilibrio, tiempo de ejecución y robustez ante el tráfico.
6. Analizar críticamente los resultados, los supuestos y las limitaciones.

1.5 Metodología general

1. **Formulación:** definición del modelo de máquinas idénticas con ventanas, almuerzo y tiempo de viaje $p_i(s_i)$.
2. **Datos:** construcción de instancias a partir de los parámetros de las 37 empresas/rutas y de supuestos experimentales explícitos.
3. **Diseño e implementación:** LS y LPT con inserción en huecos; Ramificación y Poda y CP-SAT como referencia exacta.
4. **Validación:** verificador independiente, cálculo manual y pruebas automatizadas.
5. **Experimentación:** comparación sobre las 37 rutas en tres escenarios de tráfico, contraste con el óptimo en instancias pequeñas, escalabilidad y simulación de la variabilidad del tráfico.

1.6 Alcance y delimitación

Incluye:

- las 37 empresas/rutas de referencia, cada una como instancia independiente;
- la flota operativa de cada ruta, con buses idénticos dentro de la ruta;
- la vuelta completa origen → destino → origen como trabajo indivisible;
- una jornada de operación de 06:00 a 22:00 y ventanas de inicio para cada vuelta;
- un periodo de almuerzo de 2 h por bus;
- tiempos de viaje dependientes de la hora mediante perfiles de tráfico experimentales;
- las heurísticas List Scheduling y LPT;
- un método exacto de referencia para instancias pequeñas;
- experimentación reproducible (semillas, datos y comandos documentados).

No incluye:

- cambio de buses entre rutas;
- rediseño de rutas o paraderos;
- el problema de ruteo de vehículos (VRP);
- heterogeneidad de buses;
- datos de GPS en tiempo real;
- simulación microscópica de tráfico (Lopez et al., 2018);
- modelamiento de semáforos o de la infraestructura vial.

1.7 Organización del documento

La sección 2 resume los fundamentos y el aprendizaje autónomo, y la 3 el stack tecnológico. La sección 4 formula el problema, la 5 presenta el diseño algorítmico y la 6 el diseño del sistema. La sección 7 resuelve una instancia pequeña paso a paso, la 8 describe las pruebas y la 9 la metodología experimental. La sección 10 muestra el prototipo y los resultados preliminares, la 11 los analiza y la 12 concluye.

2. Fundamentos y aprendizaje autónomo

2.1 Conceptos y algoritmos

Máquinas paralelas idénticas. En $P||C_{\max}$, n trabajos con duración p_j se reparten entre m máquinas idénticas para minimizar el makespan $C_{\max}$, el instante en que termina la última máquina (Pinedo, 2016). Como no hay restricciones temporales, cada máquina trabaja sin pausas y $C_{\max}$ coincide con la carga de la máquina más cargada. Esa equivalencia es la que permite, en la sección 4.8, usar la carga máxima $L_{\max}$ como objetivo cuando sí hay ventanas.

List Scheduling (LS) recorre los trabajos en un orden dado y asigna cada uno a la máquina menos cargada; garantiza $C_{\max} \leq (2 - 1/m) \cdot C^*$ (Graham, 1966). **LPT (Longest Processing Time first)** aplica la misma regla después de ordenar los trabajos de mayor a menor duración; garantiza $C_{\max} \leq (4/3 - 1/(3m)) \cdot C^*$ (Graham, 1969). Estas garantías suponen que cualquier trabajo puede ejecutarse en cualquier momento, por lo que no se trasladan al problema con ventanas y tráfico y deben contrastarse experimentalmente.

Ventanas e indisponibilidad. Si cada trabajo solo puede iniciar en un intervalo $[r_j, d_j]$, el problema se aproxima a la programación de intervalos, donde ya es difícil decidir si todos los trabajos caben (Kolen et al., 2007). El almuerzo convierte cada bus en una máquina con un periodo de indisponibilidad (Lee, 1996).

Tiempos dependientes del tiempo. Cuando la duración de un trabajo depende de su inicio (Gawiejnowicz, 2008), el tiempo de viaje puede calcularse dividiendo el día en franjas de velocidad constante e integrando la velocidad tramo por tramo (Ichoua, Gendreau y Potvin, 2003). Este modelo cumple la **propiedad FIFO**: si $s_1 \leq s_2$, entonces $s_1 + p(s_1) \leq s_2 + p(s_2)$.

Métodos exactos. La Ramificación y Poda (Land y Doig, 1960) explora el árbol de asignaciones y descarta las ramas cuya cota no puede mejorar la mejor solución conocida. CP-SAT, el solver de programación con restricciones de OR-Tools (Google, 2024), modela de forma nativa intervalos opcionales que no pueden solaparse. La **simulación Monte Carlo** repite la ejecución de un plan con duraciones aleatorias para estimar medias y percentiles (Law, 2015).

2.2 Fuentes y criterio de selección

Se usaron artículos con revisión por pares, libros de referencia y documentación oficial, seleccionados por su **pertinencia** para el modelo, su **confiabilidad** y la posibilidad de **verificar** lo que afirman con los propios datos o pruebas del proyecto. Las fuentes principales y su uso son:

Fuente	Uso en el proyecto
Graham (1966, 1969); Pinedo (2016); Garey y Johnson (1979)	Definición del problema, garantías de LS y LPT y complejidad.
Ichoua, Gendreau y Potvin (2003); Gawiejnowicz (2008)	Modelo de tráfico por franjas y propiedad FIFO.
Lee (1996); Kolen et al. (2007)	Modelado del almuerzo y de las ventanas de inicio.
Land y Doig (1960); documentación de OR-Tools	Métodos exactos de referencia.
Law (2015)	Diseño de la simulación Monte Carlo.
Tabla de flota operativa 2020	Parámetros de las 37 empresas/rutas (dato de referencia, no oficial).

Tabla 1. Fuentes principales y su uso.

2.3 Conocimientos adicionales y aprendizaje autónomo

El proyecto requirió conocimientos que no forman parte del contenido previo del grupo. La tabla resume qué se aprendió, para qué se usó y cómo se adquirió y verificó. Cada integrante registra sus actividades en una bitácora individual (formato en el Anexo B).

Conocimiento	Para qué se usó	Cómo se adquirió y verificó	Responsable
Tiempos de viaje por franjas y FIFO	Modelo de tráfico $p_i(s_i)$	Estudio de Ichoua et al.; cálculo manual; prueba automática de FIFO y hoja de cálculo de contraste	Aguilar
CP-SAT (intervalos opcionales, NoOverlap, AddElement)	Óptimo de referencia	Documentación oficial; modelo de prueba; contraste con Ramificación y Poda	Aguilar
Garantías de LS y LPT	Diseño y análisis de las heurísticas	Graham y Pinedo; reproducción del peor caso de LPT en una prueba	Choque
Listas ordenadas con búsqueda binaria	Línea de tiempo de cada bus	Documentación de Python; medición de escalabilidad	Choque
Cotas y simetrías en Ramificación y Poda	Método exacto propio	Land y Doig; conteo de nodos y podas; contraste con CP-SAT	Huacani
Pruebas con pytest	Casos básicos, límite y adversos	Documentación oficial; 35 pruebas automatizadas	Huacani
Validación de datos con pandas/openpyxl	Construcción de instancias	Documentación oficial; verificaciones de consistencia	Moreano
Simulación Monte Carlo	Robustez ante tráfico variable	Law (2015); réplicas con semilla fija	Moreano

Tabla 2. Conocimientos adicionales requeridos y forma de adquisición y verificación.

3. Stack tecnológico

Las herramientas se eligieron por su pertinencia para el problema, su instalación sin licencias (reproducibilidad) y su integración con Python. Se descartaron los solvers comerciales, que requieren licencia, y la simulación microscópica de tráfico, que exige una red vial calibrada que no está disponible para Cusco.

Tecnología	Función	Justificación
Python 3	Lenguaje de implementación	Lenguaje de referencia del curso; las heurísticas resuelven cada ruta en milisegundos.
Git / GitHub	Control de versiones	Historial del trabajo y de las contribuciones de cada integrante.
CSV / JSON	Instancias, perfiles de tráfico y resultados	Formatos abiertos, legibles y reproducibles.
openpyxl	Lectura y generación del Excel del proyecto	Permite leer los parámetros y producir hojas con fórmulas.
pandas / matplotlib	Análisis de resultados y gráficos	Tablas y figuras de los experimentos.
Jupyter Notebook	Demostración del prototipo	Ejecución paso a paso (también en Google Colab).
pytest	Pruebas automatizadas	Casos parametrizados y verificación continua.
OR-Tools CP-SAT	Solución exacta de referencia	Libre; modela intervalos opcionales, no solapamiento y duraciones que dependen del inicio.

Tabla 3. Stack tecnológico seleccionado.

4. Formulación del problema

4.1 Elementos del modelo

Elemento	Definición
Instancia	Una empresa/ruta. Las 37 rutas se resuelven por separado y los buses no cambian de ruta.
Máquina i	Un bus de la flota operativa de la ruta. Los buses son idénticos en velocidad; solo difieren en su turno de almuerzo.
Trabajo j	Una vuelta completa origen $\rightarrow$ destino $\rightarrow$ origen, con duración base $\bar{p}_j$ y ventana de inicio $[r_j, d_j]$.
Hora de salida s_j	Variable de decisión: instante en que inicia la vuelta j .
Tiempo de viaje $p_j(s_j)$	Duración real de la vuelta j si sale en s_j , según el perfil de tráfico (sección 4.3).
Almuerzo	Bloque de 120 min por bus que inicia en $[a_i, b_i]$: turnos de 10:30–11:30, 12:00–13:00 y 13:30–14:30 (el bus k usa el turno $k \bmod 3$).
Carga L_i	Suma de los tiempos de viaje de las vueltas asignadas al bus i .

Tabla 4. Elementos del modelo.

4.2 Origen de los datos

Tipo	Contenido
Datos de la fuente de referencia	Flota operativa, tiempo de vuelta, demanda por hora y viajes por unidad vehicular de las 37 empresas/rutas (tabla de flota operativa 2020).
Parámetros construidos por el grupo	Excel del proyecto; número de vueltas planificadas (flota $\times$ viajes por unidad, redondeado); inicios nominales equiespaciados en la jornada; instancias JSON.
Supuestos experimentales	Ventanas de inicio (± 15 min en 06:00–09:00 y 16:00–19:00; ± 30 min en el resto); jornada de 06:00 a 22:00; almuerzo de 2 h en tres turnos; perfiles de tráfico T0, T1 y T2; variabilidad σ .

Tabla 5. Clasificación de los datos del modelo.

4.3 Modelo del tráfico

$p_j(s_j)$ es el tiempo de viaje de la vuelta j si sale a la hora s_j . El día se divide en franjas $[\tau_k, \tau_{k+1})$ con un factor de congestión $f_k > 0$: dentro de la franja k , un minuto real avanza $1/f_k$ minutos del recorrido base. Así, $p_j(s_j)$ es el menor Δ que cumple:

$$\int_{s_{\{j\}}}^{s_{\{j\}} + \Delta} dt / f(t) = \bar{p}_j$$

Como f es constante por tramos, la integral se resuelve franja por franja hasta completar $\bar{p}_j$. Se definen tres perfiles:

Franja	Condición	T2 (pesimista)	T1 (normalizado)	T0 (sin tráfico)
06:00–06:30	Valle temprano	0.90	0.798	1
06:30–09:00	Punta mañana	1.30	1.152	1
09:00–12:00	Valle	1.00	0.886	1
12:00–14:30	Punta mediodía	1.20	1.064	1
14:30–17:00	Valle	1.00	0.886	1
17:00–20:00	Punta tarde	1.35	1.197	1
20:00–22:00	Valle nocturno	0.90	0.798	1
Media ponderada		1.128	1.000	1

Tabla 6. Factores de congestión por franja (parámetros experimentales). $T1 = T2 / 1.128$.



Figura 1. Perfiles de congestión T0, T1 y T2.

T1 se normaliza para que su media en la jornada sea 1. Así, la duración promedio del día coincide con el tiempo de vuelta de la fuente, que proviene de una velocidad media, y el tráfico solo redistribuye el tiempo entre puntas y valles. T2 representa un día de congestión generalizada (+12.8 % en promedio).

Variabilidad. Para evaluar la robustez, en simulación la duración realizada es $p_i(s) \cdot \varepsilon$ con $\varepsilon \sim \text{LogNormal}(-\sigma^2/2, \sigma)$, de media 1 y $\sigma \in \{0.10, 0.20\}$. Un bus no sale antes de su hora programada y, si llega tarde, la siguiente vuelta y el almuerzo se desplazan.

Aclaración. Los factores de congestión y σ son **parámetros experimentales** definidos por el grupo; no son mediciones oficiales. Se almacenan en archivos JSON y en el Excel del proyecto y pueden modificarse sin cambiar el código.

4.4 Entradas

- Ruta, empresa y flota M.
- Vueltas j con duración base $\bar{p}_j$, inicio nominal y ventana $[r_j, d_j]$.
- Jornada [06:00, 22:00] y ventana de almuerzo de cada bus.
- Perfil de tráfico (T0, T1, T2 o un archivo JSON).
- Algoritmo a ejecutar y, para la simulación, σ , número de réplicas y semilla.

4.5 Salidas

- Bus asignado, hora de salida y hora de llegada de cada vuelta, o la marca NO_ASIGNADO.

- Hora de almuerzo de cada bus.
- Vueltas atendidas, $L_{\max}$, cota inferior y brecha, desbalance y utilización.
- Resultado de la validación y tiempo de ejecución.
- En simulación: porcentaje de vueltas que salen fuera de su ventana, retraso medio y buses que terminan después de las 22:00.

4.6 Restricciones

1. **Ruta:** una vuelta solo se asigna a un bus de su propia ruta.
2. **Asignación única:** cada vuelta se asigna a lo sumo a un bus; si no se asigna, queda como no atendida.
3. **Ventana:** $r_j \leq s_j \leq d_j$.
4. **Tiempo de viaje:** la vuelta termina en $C_j = s_j + p_j(s_j)$.
5. **Jornada:** $s_j \geq 06:00$ y $C_j \leq 22:00$.
6. **No solapamiento:** dos vueltas del mismo bus no se superponen.
7. **Almuerzo:** cada bus tiene un bloque de 120 min que inicia en su ventana y no se superpone con sus vueltas.

4.7 Supuestos

1. Los buses de una ruta son idénticos en velocidad.
2. La vuelta es indivisible y su duración base es el tiempo de vuelta de la fuente.
3. En una franja, el tráfico afecta por igual a toda la ruta.
4. Las ventanas y los turnos de almuerzo son políticas experimentales reproducibles.
5. En la simulación, las perturbaciones de distintas vueltas son independientes.
6. No se consideran tiempos de terminal entre vueltas.

4.8 Función objetivo

Como hay ventanas repartidas en toda la jornada, la última vuelta termina cerca de las 22:00 cualquiera sea la asignación, así que el makespan $C_{\max}$ (hora de fin) no distingue entre soluciones. Se usa en cambio la **carga máxima de conducción** $L_{\max} = \max_i L_i$, que es la medida que $C_{\max}$ representa en $P||C_{\max}$ sin restricciones temporales (sección 2.1).

Además, en algunas rutas la flota no alcanza para todas las vueltas, de modo que el objetivo es **lexicográfico**:

1. **maximizar la cantidad de vueltas atendidas** (equivalente a minimizar U , las vueltas no asignadas);
2. **entre las soluciones con el mismo U , minimizar $L_{\max}$.**

$$\text{lex-min } (U, L_{\max}), \quad U = \sum_j u_j, \quad L_{\max} = \max_i \sum_j x_{ij} \cdot p_j(s_j)$$

Componente	Expresión
Variables	$x_{ij} \in \{0,1\}$ (vuelta j en bus i); $u_j \in \{0,1\}$ (j no atendida); $s_j \in [r_j, d_j]$; $\ell_i \in [a_i, b_i]$ (inicio del almuerzo)
Asignación	$\sum_i x_{ij} + u_j = 1 \quad \forall j$
Jornada	$s_j + p_j(s_j) \leq 22:00 \quad \forall j$
No solapamiento	los intervalos $[s_j, s_j + p_j(s_j))$ con $x_{ij} = 1$ y $[\ell_i, \ell_i + 120)$ son disjuntos $\forall i$
Carga	$L_i = \sum_j x_{ij} \cdot p_j(s_j)$; $L_{\max} \geq L_i \quad \forall i$
Objetivo (pesos)	$\min \text{BIG} \cdot U + L_{\max}$, con $\text{BIG} > m \cdot 960$ para que U tenga prioridad absoluta

Tabla 7. Formulación matemática (implementada en algorithms/cpsat.py).

Cota inferior. Sea $p_j^{\min}$ la menor duración posible de j dentro de su ventana. Para cualquier solución que atienda un conjunto S de vueltas, $L_{\max} \geq \text{LB}(S) = \max(\max_{j \in S} p_j^{\min}, \sum_{j \in S} p_j^{\min} / m)$. La brecha de una heurística se mide contra la cota de las vueltas que atiende.

4.9 Datos e instancias

El grupo construyó un Excel con los parámetros de las 37 empresas/rutas tomados de la tabla de flota operativa 2020, que es un documento de referencia y no un archivo entregado por las empresas. A partir de ese Excel se generan las instancias: una por ruta, en formato JSON, con sus buses y sus vueltas. Los perfiles de tráfico se guardan también en JSON y los resultados en CSV.

Antes de resolver una instancia se aplican controles de validación: el número de vueltas de cada ruta coincide con las vueltas planificadas, toda ventana cumple $r_j \leq d_j$, toda vuelta puede terminar dentro de la jornada (con tolerancia de 1 min por el redondeo de las horas al minuto), los identificadores son únicos y la flota no está vacía. En total hay **5 017 vueltas** (103 a 166 por ruta) y entre 16 y 44 buses por ruta.

Para anticipar la dificultad de cada ruta se define la **saturación** de la flota:

$$\rho = \sum_j \bar{p}_j / (M \cdot 14 \text{ h})$$

Si $\rho > 1$, la ruta no puede atender todas sus vueltas ni siquiera sin tráfico. Es el caso de RTU-22: 19 buses y 127 vueltas de 2.13 h requieren 270.5 h, y la flota ofrece 266 h ($\rho = 1.02$). Las rutas siguientes en saturación son RTI-02 (0.87), RTU-05 (0.82) y RTU-01 (0.79).

5. Diseño algorítmico

5.1 Estrategias

La comparación principal es **List Scheduling frente a LPT**. Ambas heurísticas siguen un esquema común de lista con inserción en huecos. Se recorre una lista de vueltas en el orden propio de cada estrategia; para cada vuelta se busca, en cada bus, la salida factible más temprana que respete la ventana, la jornada y el almuerzo, y se elige el bus de menor carga acumulada. Los métodos exactos sirven solo como referencia en instancias pequeñas.

Estrategia	Orden de la lista	Selección del bus	Rol
List Scheduling (LS)	Inicio mínimo de ventana r_i	Menor carga; empate → salida más temprana → índice	Heurística
LPT	Tiempo de viaje en la salida nominal, de mayor a menor	Igual que LS	Heurística
Ramificación y Poda	Cronológico (r_i), ramificando sobre los buses	Búsqueda exhaustiva con poda	Referencia exacta
CP-SAT	Modelo declarativo (sección 4.8)	Solver	Referencia exacta

Tabla 8. Estrategias implementadas.

5.2 Estructuras de datos

Estructura	Implementación	Operaciones y costo
Job	Registro inmutable: id, ruta, $\bar{p}$, r , d , inicio nominal	Acceso $O(1)$
Bus	Registro: id, ruta, ventana de almuerzo, duración del almuerzo	Acceso $O(1)$
TrafficProfile	Lista ordenada de franjas (inicio, fin, factor)	travel_time en $O(F)$, con $F = 7$ franjas
BusTimeline	Listas ordenadas de inicios, fines y etiquetas, más la carga del bus	Recorrido de huecos $O(k)$; inserción con $\text{bisect } O(\log k) + \text{desplazamiento } O(k)$
Schedule	Diccionario vuelta → asignación, lista de no asignadas y almuerzos	Consulta $O(1)$
Instance	Vueltas, buses e índice por identificador	Lectura y escritura JSON $O(n + m)$

Tabla 9. Estructuras de datos (k = vueltas ya asignadas al bus).

Se usa una lista ordenada con búsqueda binaria en lugar de un árbol de intervalos porque cada bus realiza a lo sumo unas seis vueltas al día. Con k tan pequeño, el costo $O(k)$ es despreciable y el código es más fácil de verificar (Cormen et al., 2022).

5.3 Pseudocódigo

Tiempo de viaje $p_j(s)$

```

función TIEMPO_VIAJE(s,  $\bar{p}$ , franjas):
    restante  $\leftarrow \bar{p}$  ;  $t \leftarrow s$ 
    para cada franja ( $\tau_{ini}$ ,  $\tau_{fin}$ , f) con  $\tau_{fin} > s$ , en orden:
        capacidad  $\leftarrow (\tau_{fin} - \max(t, \tau_{ini})) / f$   $\triangleright$  minutos base que caben en la
        franja
        si capacidad  $\geq$  restante: devolver  $\max(t, \tau_{ini}) + \text{restante} \cdot f - s$ 
        restante  $\leftarrow$  restante - capacidad ;  $t \leftarrow \tau_{fin}$ 
    devolver  $t + \text{restante} \cdot f_{fuera} - s$   $\triangleright$  después de la última franja

```

Salida factible más temprana en un bus

```

función BUSCAR_HUECO(bus, j, p):
    para cada hueco libre [ $g_0$ ,  $g_1$ ] del bus, en orden cronológico:
         $s_0 \leftarrow \max(r_j, g_0)$  ; si  $s_0 > d_j$ : devolver NINGUNO
        candidatos  $\leftarrow [s_0, \max(s_0, \max(g_0, a_{bus}) + 120)]$   $\triangleright$  salir ya, o almorzar
    antes
        para s en candidatos:
            si  $s > d_j$  o  $s + p(s) > g_1$ : romper  $\triangleright$  FIFO: más tarde tampoco
    cabe
        si el almuerzo aún cabe con  $[s, s + p(s)]$  ocupado: devolver (s,  $s + p(s)$ )
    devolver NINGUNO

```

List Scheduling y LPT

```

función GREEDY(instancia, orden, perfil):
    para cada bus i:  $L_i \leftarrow 0$  ; línea_i  $\leftarrow$  vacía
    para cada vuelta j en orden:
         $p \leftarrow t \mapsto \text{TIEMPO\_VIAJE}(t, \bar{p}_j, \text{perfil})$ 
        mejor  $\leftarrow$  NINGUNO
        para cada bus i de la ruta:
             $h \leftarrow \text{BUSCAR\_HUECO}(i, j, p)$ 
            si  $h \neq \text{NINGUNO}$  y  $(L_i, h.\text{inicio}, i) < \text{clave}(\text{mejor})$ : mejor  $\leftarrow (i, h)$ 
        si mejor = NINGUNO: marcar j como NO_ASIGNADA
        si no: insertar j en línea_i ;  $L_i \leftarrow L_i + \text{duración}$ 
    para cada bus: ubicar el almuerzo en su inicio factible más temprano

LS : orden  $\leftarrow$  vueltas ordenadas por ( $r_j$ , nominal_j)
LPT: orden  $\leftarrow$  vueltas ordenadas por  $\text{TIEMPO\_VIAJE}(\text{nominal}_j, \bar{p}_j)$  decreciente

```

Ramificación y Poda

```

función BNB(instancia):
    incumbente  $\leftarrow$  mejor(LS, LPT) ; ( $U^*$ ,  $L^*$ )  $\leftarrow$  valor(incumbente)
     $pmin_j \leftarrow \min_{t \in [r_j, d_j]} \text{TIEMPO\_VIAJE}(t, \bar{p}_j)$  ;  $\text{suf}_k \leftarrow \sum_{q \geq k} pmin_q$ 
    RAMIFICAR( $k = 0$ ,  $U = 0$ , total = 0)

función RAMIFICAR(k, U, total):
    LB  $\leftarrow \max(\max_i L_i, (total + \text{suf}_k) / m)$ 
    si  $(U, LB) \geq (U^*, L^*)$ : podar
    si  $k = n$ : actualizar incumbente ; volver
    para cada bus i (omitiendo buses vacíos con el mismo turno de almuerzo):
         $h \leftarrow \text{BUSCAR\_HUECO}(i, j_k)$  ; si h: insertar, RAMIFICAR( $k+1$ , U, total +
        duración), deshacer
    si  $U + 1 \leq U^*$ : RAMIFICAR( $k+1$ ,  $U+1$ , total)  $\triangleright$  dejar  $j_k$  sin atender

```

La Ramificación y Poda es exacta dentro del espacio de programas en que cada bus ejecuta sus vueltas en orden cronológico y con la salida más temprana posible, el mismo espacio que recorre LS. CP-SAT es exacto sobre el modelo completo, porque puede retrasar una salida dentro de su ventana; la diferencia entre ambos mide el valor de elegir la hora de salida.

5.4 Invariantes y corrección

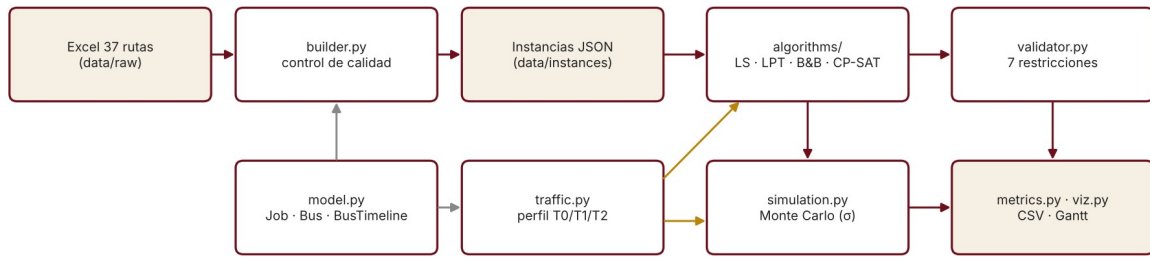
- **I1 — Factibilidad parcial.** Tras cada inserción, la línea de tiempo de cada bus cumple ventana, jornada y no solapamiento, y conserva un hueco donde cabe el almuerzo. BUSCAR_HUECO solo acepta candidatos que mantienen esta condición, así que el invariante se conserva por inducción sobre las inserciones.
- **I2 — Salida más temprana.** Por la propiedad FIFO, $s \mapsto s + p(s)$ es no decreciente. Dentro de un hueco, el candidato más temprano termina antes que cualquier otro posterior; si ese no cabe, ninguno posterior cabe.
- **I3 — Conservación.** Toda vuelta termina asignada a un solo bus o registrada como no atendida.
- **I4 — Cota válida.** Cada vuelta dura al menos $p^{\min}_j$ y la carga total se reparte entre m buses, de modo que $LB \leq L_{\max}$. La Ramificación y Poda solo poda ramas que ni con la cota pueden mejorar al incumbente.

5.5 Complejidad

Componente	Tiempo	Espacio	Comentario
TIEMPO_VIAJE	$O(F)$	$O(1)$	$F = 7$ franjas
BUSCAR_HUECO	$O(k)$	$O(k)$	$k \approx n/m$ vueltas por bus
LS y LPT	$O(n \log n + n \cdot m \cdot k)$	$O(n + m)$	Con $k \approx n/m$ queda $O(n^2)$; ≈ 16 ms por ruta real
Ramificación y Poda	$O(m^n)$ en el peor caso	$O(n + m)$	Con poda y simetría; usado con $n \leq 12$
CP-SAT	Exponencial en el peor caso	$O(n \cdot m)$	$n \cdot m$ variables binarias
Validador	$O(n \log n)$	$O(n)$	Ordena por bus y revisa vecinos
Simulación	$O(R \cdot n)$	$O(n)$	R réplicas

Tabla 10. Complejidad de los componentes.

6. Diseño del sistema



Flechas guinda: datos de la instancia · doradas: modelo de tráfico · grises: estructuras compartidas

Figura 2. Módulos del sistema y flujo de información.

El Excel del proyecto se transforma en instancias JSON validadas. Cada instancia se resuelve con el algoritmo y el perfil de tráfico elegidos, el validador revisa el resultado y se generan métricas, archivos CSV y diagramas de Gantt. La simulación toma un plan y lo ejecuta con tráfico variable.

Módulo	Responsabilidad
traffic.py	Perfiles T0/T1/T2 y cálculo de $p(s)$.
model.py	Vueltas, buses, instancia, línea de tiempo por bus y programa.
builder.py	Lectura del Excel, validación de datos, instancia del ejemplo manual y generador aleatorio con semilla.
algorithms/	greedy.py (LS y LPT), bnb.py (Ramificación y Poda), cpsat.py (CP-SAT).
validator.py	Verificación independiente de todas las restricciones.
metrics.py	Vueltas atendidas, L_{max} , cota inferior y brecha, desbalance, utilización, saturación p .
simulation.py	Simulación Monte Carlo de la ejecución de un plan.
viz.py, cli.py	Diagramas de Gantt e interfaz de línea de comandos.

Tabla 11. Módulos del paquete sched.

Formatos. Las instancias se guardan en JSON, una por ruta, con horas en formato HH:MM y duraciones en minutos:

```

{"route": "RTI-01", "company": "E.T. SAYLLA S.A.", "day_start": "06:00", "day_end": "22:00",
 "buses": [{"id": "RTI-01-B1", "lunch_r": "10:30", "lunch_d": "11:30", "lunch_min": 120}, ...],
 "jobs": [{"id": "RTI-01-J001", "base_min": 148.8, "r": "06:00", "d": "06:15", "nominal": "06:00"}, ...]}
  
```

Los perfiles de tráfico usan JSON con franjas (inicio, fin, factor). La salida es un CSV con una fila por vuelta: ruta, vuelta, bus, ventana, salida, llegada, duración con tráfico y duración base; las vueltas no atendidas llevan bus = NO_ASIGNADO. Todas las rutas de archivo son relativas al repositorio.

7. Ejemplo manual

7.1 Datos

Se usa una ruta pequeña con duración base de **90 min** (cercana a la de RTI-08, 1.57 h, y redondeada para el cálculo a mano). Tiene 3 buses con almuerzo de 2 h (B1 inicia entre 10:30 y 11:30, B2 entre 12:00 y 13:00 y B3 entre 13:30 y 14:30), 8 vueltas y el perfil de tráfico **T2**, que tiene factores redondos.

Vuelta	Ventana de inicio	p_i si sale en r_i	p_i si sale en d_i	p_i en la salida nominal
J1	06:00–06:30	103.7	117.0	110.3
J2	06:30–07:00	117.0	117.0	117.0
J3	07:00–07:30	117.0	110.8	114.2
J4	08:00–08:30	103.8	96.9	100.4
J5	09:30–10:30	90.0	90.0	90.0
J6	11:30–12:30	102.0	108.0	108.0
J7	16:30–17:00	111.0	121.5	116.3
J8	17:00–17:30	121.5	121.5	121.5

Tabla 12. Vueltas de la instancia manual y su tiempo de viaje (min) con tráfico T2.

7.2 Cálculo del tiempo de viaje con tráfico

J1 saliendo a las 06:00. La franja 06:00–06:30 ($f = 0.9$) dura 30 min reales y cubre $30/0.9 = 33.33$ min base. Faltan 56.67 min base, que en la punta mañana ($f = 1.3$) toman $56.67 \times 1.3 = 73.67$ min. Entonces $p_{J1}(06:00) = 30 + 73.67 = \mathbf{103.67 \text{ min}}$.

J4 saliendo a las 08:00. Hasta las 09:00 hay 60 min reales a $f = 1.3$, que cubren 46.15 min base; los 43.85 restantes van a $f = 1.0$. Entonces $p_{J4}(08:00) = 60 + 43.85 = \mathbf{103.85 \text{ min}}$.

J3 saliendo a las 07:00 o a las 07:29. A las 07:00 recorre toda la vuelta en punta: $90 \times 1.3 = \mathbf{117.0 \text{ min}}$. A las 07:29, los 91 min reales hasta las 09:00 cubren 70 min base y los 20 restantes van a $f = 1.0$: **111.0 min**. Salir 29 min más tarde ahorra 6 min de conducción.

Verificación. La hoja *Calculadora vuelta* del Excel reproduce este cálculo con fórmulas (06:00, 90 min, T2 $\rightarrow$ 103.67 min) y coincide con el programa.

7.3 List Scheduling

Orden: J1, J2, ..., J8 (por inicio de ventana). En cada paso se muestra, para cada bus, la salida y llegada factibles y la carga previa (min).

Paso	Vuelta	B1	B2	B3	Elegido
1	J1	06:00–07:44 · 0	06:00–07:44 · 0	06:00–07:44 · 0	B1 (empate → índice)
2	J2	infactible	06:30–08:27 · 0	06:30–08:27 · 0	B2
3	J3	infactible	infactible	07:00–08:57 · 0	B3
4	J4	08:00–09:44 · 103.7	08:27–10:05 · 117.0	infactible	B1 (menor carga)
5	J5	09:44–11:14 · 207.5	09:30–11:00 · 117.0	09:30–11:00 · 117.0	B2 (empate → índice)
6	J6	12:30–14:18 · 207.5 (*)	infactible	11:30–13:12 · 117.0	B3
7	J7	16:30–18:21 · 207.5	16:30–18:21 · 207.0	16:30–18:21 · 219.0	B2 (207.0 < 207.5)
8	J8	17:00–19:02 · 207.5	infactible	17:00–19:02 · 219.0	B1

Tabla 13. Traza de List Scheduling (T2). (*) B1 debe iniciar su almuerzo entre 10:30 y 11:30, por eso J6 solo cabe después.

Resultado: $B1 = 103.7 + 103.8 + 121.5 = \mathbf{329.0 \text{ min}}$; $B2 = 117.0 + 90.0 + 111.0 = 318.0$; $B3 = 117.0 + 102.0 = 219.0$. Las 8 vueltas se atienden y $L_{\max} = \mathbf{329.0 \text{ min}}$.

7.4 LPT

Ordenando por tiempo de viaje en la salida nominal (última columna de la tabla de datos), el orden es J8 (121.5), J2 (117.0), J7 (116.3), J3 (114.2), J1 (110.3), J6 (108.0), J4 (100.4) y J5 (90.0).

Paso	Vuelta	Elegido	Salida–llegada	p_i	Motivo
1	J8	B1	17:00–19:02	121.5	Todos vacíos → índice
2	J2	B2	06:30–08:27	117.0	B2 y B3 con carga 0
3	J7	B3	16:30–18:21	111.0	B1 ocupado por J8; B3 con carga 0
4	J3	B3	07:00–08:57	117.0	B3 (111.0) < B1 (121.5); B2 ocupado
5	J1	B1	06:00–07:44	103.7	Único factible
6	J6	B1	12:30–14:18	108.0	B1 (225.2) < B3 (228.0)
7	J4	B2	08:27–10:05	97.6	B2 (117.0) es el de menor carga
8	J5	B2	10:05–11:35	90.0	B1 infactible; B2 (214.6) < B3 (228.0)

Tabla 14. Traza de LPT (T2).

Resultado: $B1 = 121.5 + 103.7 + 108.0 = \mathbf{333.2 \text{ min}}$, $B2 = 304.6$ y $B3 = 228.0$. Se atienden las 8 vueltas con $L_{\max} = \mathbf{333.2 \text{ min}}$, peor que LS. Al tomar primero las vueltas largas de la tarde, LPT reparte las puntas sin seguir el orden temporal, y al final J6 solo cabe en B1 después de su almuerzo.

7.5 Resultados y comparación con la referencia exacta

Método	B1	B2	B3	$L_{\max}$ (min)	Brecha vs óptimo
Cota inferior LB	—	—	—	284.3	—
LS	329.0	318.0	219.0	329.0	+5.4 %
LPT	333.2	304.6	228.0	333.2	+6.8 %
Ramificación y Poda (salida más temprana)	315.2	228.0	316.6	316.6	+1.5 %
CP-SAT (óptimo)	232.5	311.6	312.0	312.0	0 %
Sin tráfico (T0): LS = LPT = exactos				270.0	0 %

Tabla 15. Comparación en la instancia manual (cargas en minutos).

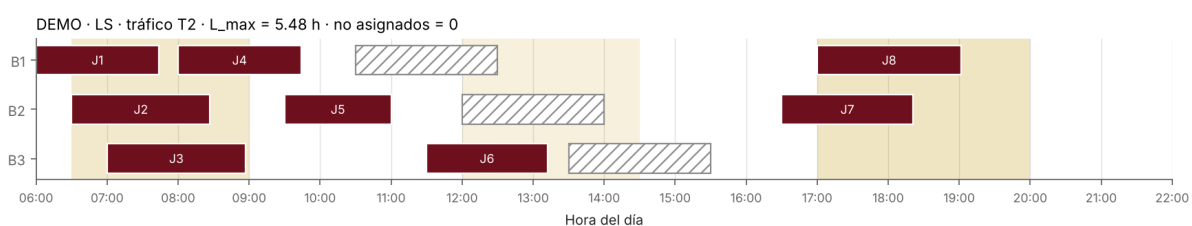


Figura 3. Programa de List Scheduling (T2). Fondo dorado: franjas de congestión; rayado: almuerzo.

Sin tráfico, todas las estrategias obtienen 270 min (3 + 3 + 2 vueltas de 90 min). Con tráfico, el orden de procesamiento importa y LS supera a LPT. El óptimo de CP-SAT retrasa J3 hasta las 07:29 para evitar parte de la punta, lo que confirma que la hora de salida influye en la carga.

8. Pruebas y validación

8.1 Validación de restricciones

El módulo `validator.py` verifica cada solución sin usar las estructuras internas de los algoritmos. Recalcula: (1) que cada vuelta aparezca una sola vez, asignada o no atendida; (2) que el bus sea de la ruta; (3) la ventana; (4) la jornada; (5) que la duración coincida con $p_i(s_i)$; (6) que el almuerzo sea único, de 120 min y esté dentro de su ventana; y (7) que no haya solapamientos. Cada ejecución desde la línea de comandos lo aplica y termina con error si encuentra violaciones. Todas las soluciones de los experimentos se validaron, con 0 violaciones.

8.2 Casos de prueba

Tipo	Caso	Qué verifica
Básico	Instancia manual con LS, LPT y Ramificación y Poda en T0, T1 y T2	Soluciones válidas y completas
Básico	Resultados conocidos de la instancia manual	270 / 329.0 / 333.2 / 316.6 min
Básico	Cálculo de $p_{j1}(06:00)$ en T2	$30 + 56.67 \times 1.3$ min
Límite	Ventana de ancho cero ($r_j = d_j$)	Salida exacta en r_j
Límite	Vuelta que termina exactamente a las 22:00	Se acepta
Límite	Un solo bus ($m = 1$)	Solución válida
Límite	Duraciones iguales sin tráfico	LPT coincide con LS
Adverso	3 vueltas simultáneas y 1 bus	2 vueltas no atendidas, sin violaciones
Adverso	Vuelta que choca con un almuerzo fijo	Queda no atendida
Adverso	Vuelta que choca con un almuerzo flexible	El almuerzo se desplaza
Adverso	Peor caso clásico de LPT $\{3, 3, 2, 2, 2\}$, $m = 2$	LPT = 7 h; óptimo = 6 h
Adverso	Solución corrompida a propósito	El validador la rechaza
Escala	400 vueltas y 60 buses	Solución válida; todas las vueltas contabilizadas
Validación	Propiedad FIFO en 4 000 pares aleatorios	$s_1 \leq s_2 \Rightarrow \text{llegada}_1 \leq \text{llegada}_2$
Validación	Lectura del Excel y control de datos	37 rutas, 5 017 vueltas, ventanas válidas
Validación	Simulación con $\sigma = 0$ y semilla fija	Reproduce el plan; resultados repetibles
Comparación exacta	Ramificación y Poda frente a LS y LPT (16 instancias)	Nunca peor que las heurísticas
Comparación exacta	CP-SAT frente a Ramificación y Poda (T0 y T2)	CP-SAT óptimo y $\leq$ Ramificación y Poda
Comparación exacta	Cota inferior frente al óptimo	$LB \leq L_{\max}$ óptimo

Tabla 16. Casos de prueba automatizados (35 pruebas con pytest, todas aprobadas).

8.3 Verificación de las salidas

1. **Factibilidad:** el validador recalcula todas las restricciones.
2. **Optimalidad en instancias pequeñas:** comparación con Ramificación y Poda y CP-SAT, dos métodos exactos independientes que coinciden en 43 de 45 instancias sin tráfico. En las otras 2, la Ramificación y Poda es ligeramente mejor (−1.8 % y −0.7 %) porque CP-SAT redondea las duraciones al minuto.

3. **Calidad en instancias grandes:** brecha respecto de la cota inferior.
4. **Cálculo a mano:** la instancia de la sección 7 y la calculadora del Excel coinciden con el programa.
5. **Instancias nuevas:** cualquier instancia escrita en el formato de la sección 6 se resuelve y valida con `python -m sched run --instance archivo.json`.

9. Metodología experimental

9.1 Preguntas e hipótesis

#	Pregunta	Hipótesis
Q1	¿Qué estrategia atiende más vueltas y obtiene menor $L_{\max}$?	H1: sin tráfico LS y LPT coinciden (las vueltas de una ruta duran lo mismo); con tráfico LS supera a LPT.
Q2	¿Cómo afecta el tráfico a la cantidad de vueltas atendidas?	H2: T1 tiene un efecto moderado; T2 reduce marcadamente la atención en rutas de alta saturación.
Q3	¿Qué tan cerca están las heurísticas del óptimo?	H3: brecha media de LS menor al 15 % en instancias pequeñas.
Q4	¿Cómo crece el tiempo de ejecución?	H4: crecimiento aproximadamente cuadrático, con milisegundos por ruta real.
Q5	¿Qué tan robusto es un plan ante tráfico variable?	H5: con $\sigma = 0.20$ más del 5 % de las vueltas sale fuera de su ventana.

Tabla 17. Preguntas de evaluación e hipótesis.

9.2 Experimentos, instancias y variables

Exp.	Instancias	Variables (factores)	Métricas
E1	37 rutas (5 017 vueltas)	Algoritmo {LS, LPT} $\times$ tráfico {T0, T1, T2}	Vueltas no atendidas, $L_{\max}$, brecha vs LB, desbalance, sobrecosto por tráfico, tiempo
E2	90 instancias aleatorias: $n \in \{8, 10, 12\}$, $m = 3, 15$ semillas, duración base $\pm 25\%$	Algoritmo {LS, LPT, Ramificación y Poda, CP-SAT} $\times$ {T0, T2}	Brecha vs óptimo, óptimos alcanzados, vueltas atendidas, tiempo
E3	Sintéticas, n de 100 a 4 000, $m = n/3$	Algoritmo {LS, LPT}, T1	Tiempo de ejecución (mejor de 3)
E4	37 rutas, 100 réplicas por ruta	Algoritmo $\times \sigma \{0.10, 0.20\}$, T1	% de vueltas fuera de ventana (media y p95), retraso medio, % de buses con horas extra

Tabla 18. Diseño experimental.

Métricas principales (función objetivo): vueltas no atendidas U y carga máxima $L_{\max}$. **Métricas complementarias:** brecha de $L_{\max}$ respecto de la cota inferior de las vueltas atendidas, desbalance $L_{\max} - L_{\min}$, sobrecosto por tráfico $(\sum p_j(s_j) - \sum \bar{p}_j) / \sum \bar{p}_j$, tiempo de ejecución y , y para la robustez, las métricas de la simulación.

9.3 Entorno y procedimiento reproducible

- Hardware: Intel Xeon 2.10 GHz (2 núcleos), Linux. Software: Python 3.13, pandas 3.0, matplotlib 3.11, openpyxl 3.1, OR-Tools 9.15, pytest.
- Procedimiento: `python -m sched build` genera las instancias; `python experiments/run_experiments.py` ejecuta E1–E4 y escribe CSV y figuras en `results/`; `python experiments/build_excel.py` actualiza el Excel.
- Semillas fijas: generador de instancias, simulación (2026) y CP-SAT (2026).

10. Prototipo y resultados preliminares

10.1 Prototipo

Repositorio: [completar con la URL de GitHub del grupo]

```

transporte-cusco-scheduling/
├── src/sched/          traffic · model · builder · validator · metrics · simulation ·
viz · cli
│   └── algorithms/    greedy (LS, LPT) · bnb · cpsat
├── tests/              35 pruebas (pytest)
├── experiments/        run_experiments.py · build_excel.py · demo_prototipo.ipynb
├── data/               raw/ (Excel) · instances/ (JSON) · traffic/ (perfiles JSON)
├── results/            CSV, figuras y Excel actualizado
└── docs/               plan de aprendizaje, bitácoras y matriz de contribuciones

```

El prototipo **carga una instancia, ejecuta un algoritmo, produce una salida verificable y la valida:**

```

$ python -m sched build
37 instancias + DEMO escritas en data/instances; perfiles en data/traffic

$ python -m sched run --route RTI-01 --alg ls --traffic T1 --gantt
RTI-01 (E.T. SAYLLA S.A.) · 35 buses · 103 vueltas · algoritmo LS · tráfico T1
  L_max = 7.772 h | L_min = 5.207 h | desbalance = 2.565 h | cota inf. = 7.063 h | gap
= 10.0 %
  asignados = 103/103 | utilización = 52.5 % | sobrecosto tráfico = 0.7 % | tiempo =
12.5 ms
  VALIDACIÓN: OK (0 violaciones)
  asignaciones -> results/RTI-01_LS_T1.csv

$ python -m sched demo --traffic T2      # instancia manual con LS, LPT y referencias
exactas
$ python -m pytest -q
35 passed

```

Una instancia conocida (la de la sección 7) o una nueva escrita en el formato JSON de la sección 6 se resuelve con `python -m sched run --instance archivo.json --alg ls`; el validador confirma la factibilidad y `--alg bnb` o `--alg cpsat` da el óptimo de referencia.

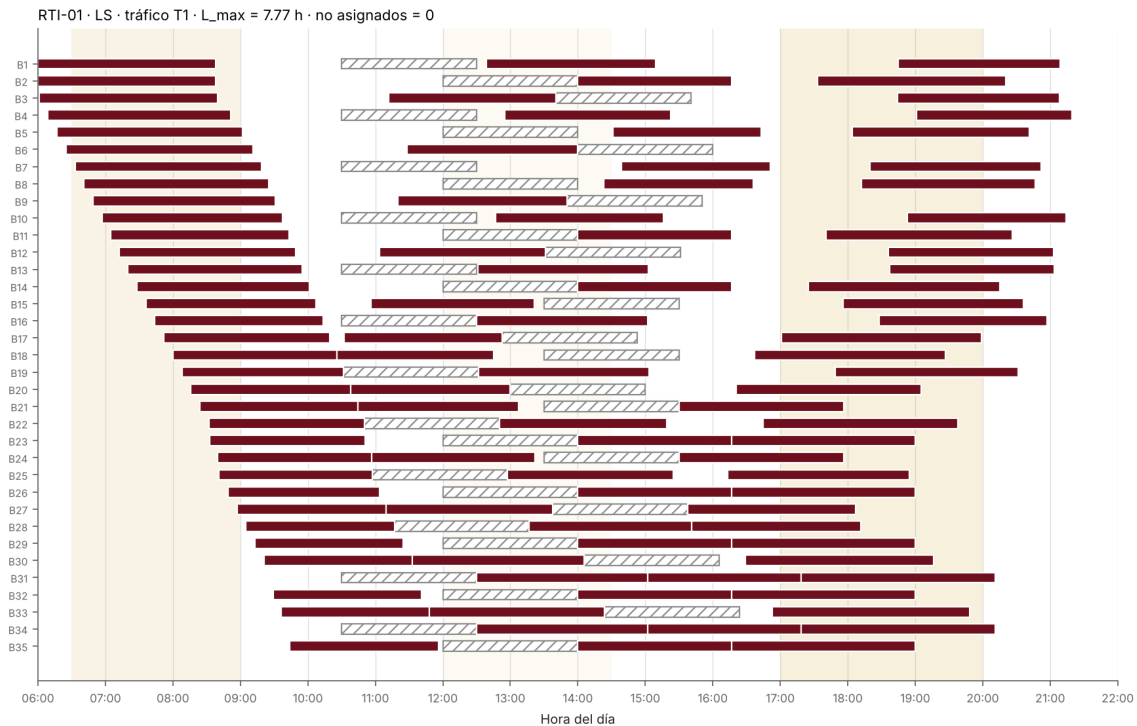


Figura 4. Programa de RTI-01 (35 buses, 103 vueltas) con List Scheduling y tráfico T1.

10.2 E1 — Las 37 rutas

Tráfico	Algoritmo	Vueltas no atendidas	Rutas completas	$L_{\max}$ medio (h)	Brecha vs LB	Desbalance (h)	Sobrecosto por tráfico
T0	LS = LPT	95 (1.9 %)	18	9.23	10.3 %	1.92	0 %
T1	LS	167 (3.3 %)	8	9.05	13.7 %	1.68	+0.8 %
T1	LPT	331 (6.6 %)	10	9.22	19.9 %	2.61	+0.8 %
T2	LS	280 (5.6 %)	4	9.97	14.5 %	1.94	+13.3 %
T2	LPT	603 (12.0 %)	4	10.08	24.1 %	3.47	+13.7 %

Tabla 19. Resultados agregados de E1 (5 017 vueltas; brecha y desbalance promediados por ruta; 0 violaciones).

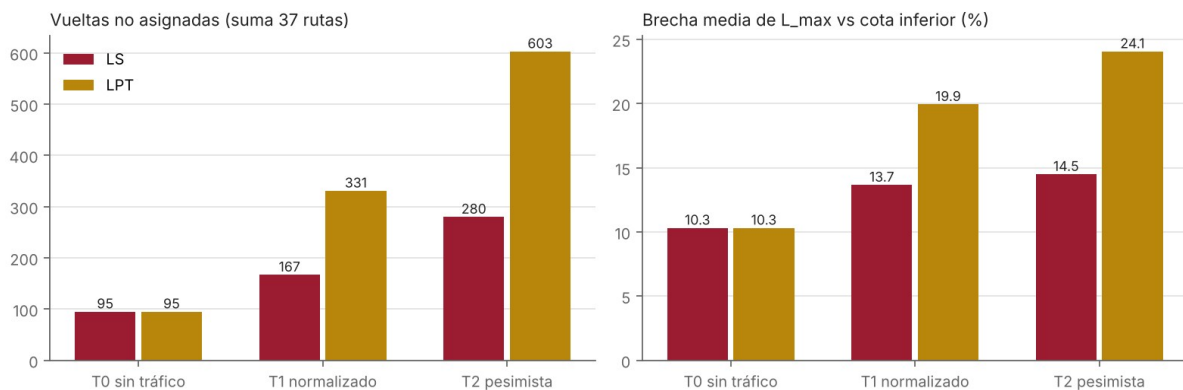


Figura 5. Vueltas no atendidas y brecha media respecto de la cota inferior, por escenario de tráfico.

- **Sin tráfico, LS y LPT coinciden.** Las vueltas de una ruta tienen la misma duración base, así que el orden de LPT empata y se desempata por r_i , que es el orden de LS.
- **Con tráfico, LS supera a LPT.** En T1 deja la mitad de vueltas sin atender (167 frente a 331) y es igual o mejor en 28 de 37 rutas; en T2 es mejor en 33 de 37. Su desbalance es 36–44 % menor.
- **El tráfico reduce la atención:** con LS pasa de 98.1 % (T0) a 96.7 % (T1) y 94.4 % (T2). En T1 la conducción total casi no cambia (+0.8 %) porque el perfil solo redistribuye el tiempo; en T2 aumenta 13.3 %.
- **Las vueltas no atendidas sin tráfico se concentran en rutas saturadas:** RTU-22 (21; $\rho = 1.02$), RTI-02 (15; $\rho = 0.87$), RTU-01 (12; $\rho = 0.79$) y RTU-05 (7; $\rho = 0.82$).
- **Rendimiento:** cada algoritmo resuelve las 37 rutas en unos 0.6 s (≈ 16 ms por ruta).

10.3 E2 — Comparación con el óptimo en instancias pequeñas

Tráfico	Algoritmo	Brecha media vs óptimo	Brecha máx.	Óptimo alcanzado	Atiende menos que el óptimo	Tiempo medio
T0	LS	8.1 %	44.4 %	4 / 45	9 / 45	0.2 ms
T0	LPT	8.7 %	35.3 %	2 / 45	26 / 45	0.2 ms
T0	Ramificación y Poda	−0.06 %	0 %	43 / 45	0 / 45	22 ms
T2	LS	10.4 %	34.8 %	0 / 45	12 / 45	0.2 ms
T2	LPT	17.6 %	59.6 %	0 / 45	32 / 45	0.2 ms
T2	Ramificación y Poda	2.0 %	6.2 %	8 / 45	0 / 45	21 ms

Tabla 20. E2: 45 instancias por escenario ($n \in \{8, 10, 12\}$, $m = 3$). Óptimo = CP-SAT (estado OPTIMAL en las 90; 45 ms en T0 y 289 ms en T2). La brecha se promedia donde el número de vueltas atendidas coincide con el óptimo.

LS queda en promedio a 8–10 % del óptimo y atiende menos vueltas que el óptimo en el 20–27 % de las instancias; LPT en el 58–71 %. Con tráfico, la Ramificación y Poda (salida más temprana) queda a 2.0 % del óptimo de CP-SAT: esa diferencia es el valor de elegir la hora de salida dentro de la ventana.

10.4 E3 — Escalabilidad

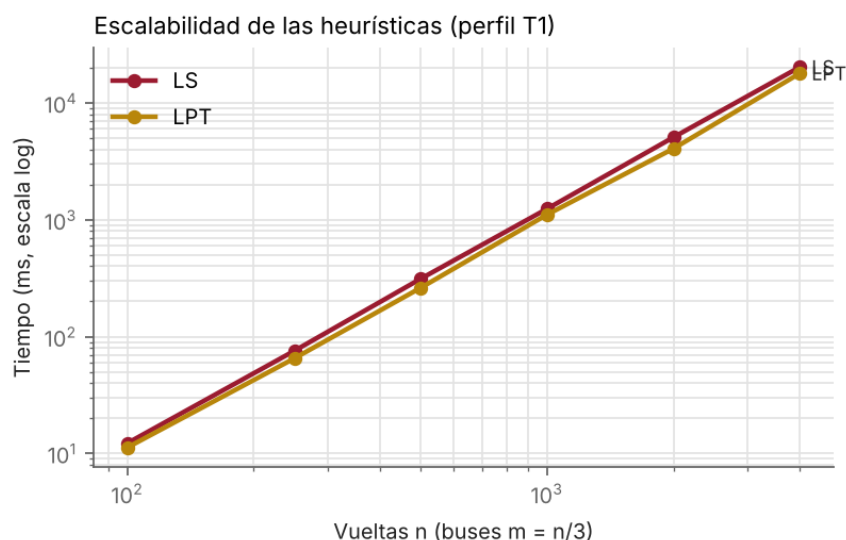


Figura 6. Tiempo de ejecución de LS y LPT según el número de vueltas (escala logarítmica).

La pendiente en escala logarítmica es cercana a 2: al duplicar n el tiempo se multiplica por unas 4 (de 5.1 s a 20.3 s entre $n = 2\,000$ y $n = 4\,000$), lo que concuerda con el análisis $O(n^2)$. Las rutas reales, de hasta 166 vueltas, se resuelven en milisegundos.

10.5 E4 — Robustez ante tráfico variable

Algoritmo	σ	% vueltas fuera de ventana (media / p95)	Retraso medio (min)	% buses con horas extra	Vueltas no atendidas en el plan
LS	0.10	6.0 / 9.2	1.7	0.2	167
LS	0.20	8.8 / 12.7	4.1	1.7	167
LPT	0.10	2.6 / 4.6	1.2	0.2	331
LPT	0.20	5.0 / 8.2	3.2	2.0	331

Tabla 21. E4: simulación Monte Carlo con perfil T1, 100 réplicas por ruta.

Con variabilidad alta ($\sigma = 0.20$), el 8.8 % de las vueltas de LS sale fuera de su ventana y el 1.7 % de los buses termina después de las 22:00. LPT muestra menos retrasos, pero atiende la mitad de vueltas menos y por eso sus buses tienen más tiempo libre; la comparación de robustez solo es justa con el mismo número de vueltas atendidas.

10.6 Contraste con las hipótesis

Hipótesis	Resultado preliminar	Estado
H1: LS = LPT sin tráfico; LS mejor con tráfico	Idénticos en T0; LS mejor en T1 y T2	Confirmada
H2: efecto moderado de T1 y marcado de T2	+72 vueltas no atendidas en T1 y +185 en T2 respecto de T0	Confirmada
H3: brecha media de LS < 15 %	8.1 % (T0) y 10.4 % (T2)	Confirmada
H4: crecimiento $\approx$ cuadrático	Pendiente logarítmica $\approx$ 2	Confirmada
H5: > 5 % fuera de ventana con $\sigma = 0.20$	8.8 % con LS	Confirmada

Tabla 22. Contraste de hipótesis.

11. Discusión y limitaciones

11.1 Interpretación

La garantía teórica de LPT supone que cualquier trabajo puede ejecutarse en cualquier momento, de modo que solo importa su tamaño. Con ventanas de inicio, en cambio, el orden de procesamiento determina la factibilidad. Cuando las ventanas son estrechas, el problema se parece a la partición de intervalos, donde procesar los intervalos por hora de inicio es la estrategia óptima en número de máquinas (Kolen et al., 2007). LS conserva ese orden temporal; LPT lo rompe al insertar primero vueltas de la tarde y dejar huecos fragmentados en la mañana.

Con tráfico, además, la vuelta "más larga" es la que cae en hora punta, así que LPT termina ordenando por congestión y no por tamaño, y concentra vueltas de punta en los mismos buses. El mayor desbalance de LPT lo confirma (2.61 h frente a 1.68 h en T1).

Por último, tanto CP-SAT como la diferencia entre la Ramificación y Poda y el óptimo muestran que elegir **cuándo** sale una vuelta, no solo en qué bus, reduce la carga en alrededor de 1.5–2 %. Este efecto existe solo porque el modelo considera tiempos de viaje dependientes de la hora.

11.2 Limitaciones y dificultades pendientes

- **Parámetros experimentales.** Las ventanas, los turnos de almuerzo y los factores de tráfico son supuestos; los resultados comparan algoritmos y no predicen la operación real. Calibrar el perfil de tráfico con mediciones de campo sigue pendiente.
- **Datos de referencia.** La tabla de 2020 es un documento secundario y representa una jornada típica.
- **Brecha de las heurísticas.** LS queda a 8–14 % de la cota o del óptimo; reducir esa brecha sin perder rapidez sigue siendo una dificultad abierta.
- **Métodos exactos.** CP-SAT no demuestra optimalidad en rutas completas en 2 minutos, y la Ramificación y Poda solo explora salidas tempranas.
- **Comparación de robustez.** E4 compara planes que atienden distinto número de vueltas, lo que favorece a LPT.
- **Discretización.** CP-SAT trabaja con minutos enteros; las soluciones se validan con las duraciones exactas.

12. Conclusiones

1. Se formuló la asignación de vueltas a buses como un problema de máquinas paralelas idénticas con ventanas de inicio, almuerzo y tiempos de viaje $p_i(s_i)$ que dependen de la hora de salida. El objetivo es lexicográfico: primero maximizar las vueltas atendidas y luego minimizar la carga máxima $L_{\max}$.
2. El tráfico se incorporó mediante un perfil de congestión por franjas que cumple la propiedad FIFO, evaluado en tres escenarios experimentales y con simulación de la variabilidad.
3. List Scheduling es la estrategia más adecuada en este contexto. Sin tráfico iguala a LPT y con tráfico atiende más vueltas (en T1 deja 167 sin atender frente a 331) y equilibra mejor la carga.
4. El tráfico tiene un efecto medible: con el perfil pesimista, la atención con LS baja de 98.1 % a 94.4 % y la conducción total aumenta 13 %.
5. En instancias pequeñas, LS queda en promedio a 8–10 % del óptimo, y elegir la hora de salida dentro de la ventana mejora la carga en alrededor de 2 %.
6. La saturación de la flota (ρ) explica la mayoría de las vueltas no atendidas: en RTU-22 la demanda de horas supera la capacidad de la flota ($\rho = 1.02$).
7. El prototipo es reproducible: instancias en JSON, algoritmos, validador independiente, 35 pruebas automatizadas, experimentos con semillas fijas y un Excel cuyas fórmulas replican el modelo de tráfico.

13. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press.
- Garey, M. R. y Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman.
- Gawiejnowicz, S. (2008). *Time-dependent scheduling*. Springer.
- Google. (2024). *OR-Tools: CP-SAT solver* [Documentación oficial].
<https://developers.google.com/optimization>
- Graham, R. L. (1966). Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45(9), 1563–1581.
- Graham, R. L. (1969). Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2), 416–429.
- Ibarra-Rojas, O. J., Delgado, F., Giesen, R. y Muñoz, J. C. (2015). Planning, operation, and control of bus transport systems: A literature review. *Transportation Research Part B: Methodological*, 77, 38–75.
- Ichoua, S., Gendreau, M. y Potvin, J.-Y. (2003). Vehicle dispatching with time-dependent travel times. *European Journal of Operational Research*, 144(2), 379–396.
- Kolen, A. W. J., Lenstra, J. K., Papadimitriou, C. H. y Spieksma, F. C. R. (2007). Interval scheduling: A survey. *Naval Research Logistics*, 54(5), 530–543.
- Land, A. H. y Doig, A. G. (1960). An automatic method of solving discrete programming problems. *Econometrica*, 28(3), 497–520.
- Law, A. M. (2015). *Simulation modeling and analysis* (5.^a ed.). McGraw-Hill.
- Lee, C.-Y. (1996). Machine scheduling with an availability constraint. *Journal of Global Optimization*, 9, 395–416.
- Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P. y Wießner, E. (2018). Microscopic traffic simulation using SUMO. En *21st IEEE International Conference on Intelligent Transportation Systems (ITSC)* (pp. 2575–2582).
- Pinedo, M. L. (2016). *Scheduling: Theory, algorithms, and systems* (5.^a ed.). Springer.
- Tabla de datos – Unidades de transporte – Flota operativa 2020. (s. f.). Scribd.
<https://es.scribd.com/document/553897271/TABLA-DE-DATOS-UNIDADES-DE-TRANSPORTE-FLOTA-OPERATIVA-2020>

Anexo A. Manual de instalación y uso

Requisitos: Python ≥ 3.10 , en Windows, Linux o macOS.

```
git clone <URL-del-repositorio> && cd transporte-cusco-scheduling
python -m venv .venv
source .venv/bin/activate                # Windows: .venv\Scripts\activate
pip install -r requirements.txt
pip install -e .

python -m sched build                    # genera data/instances/*.json
python -m sched profiles                 # muestra los perfiles T0, T1 y T2
python -m sched demo --traffic T2       # instancia manual
python -m sched run --route RTU-05 --alg lpt --traffic T1 --gantt
python -m sched simulate --route RTI-01 --alg ls --traffic T1 --sigma 0.2
python -m pytest -q                     # 35 pruebas
python experiments/run_experiments.py   # experimentos E1-E4
python experiments/build_excel.py       # Excel actualizado
```

Opción	Valores	Efecto
--route / --instance	código de ruta, DEMO o archivo JSON	Instancia a resolver
--alg	ls · lpt · bnb · cpsat	Algoritmo
--traffic	T0 · T1 · T2 · archivo.json	Perfil de tráfico
--sigma, --reps, --seed	p. ej. 0.2, 200, 2026	Parámetros de la simulación
--gantt, --out	—	Figura y ruta del CSV de salida

Tabla 23. Opciones de la línea de comandos.

Anexo B. Contribuciones y bitácora

R = responsable, A = apoyo, V = revisión. La distribución se contrasta con el historial del repositorio.

Componente	Aguilar	Choque	Huacani	Moreano
Formulación y modelo de tráfico	R	V	V	A
Modelo de datos y línea de tiempo	R	A	V	
List Scheduling y LPT	A	R	V	
Ramificación y Poda	V	A	R	
CP-SAT	R		A	
Validador, métricas y pruebas	A	A	R	A
Datos, Excel e instancias	V			R
Simulación, experimentos y gráficos	A		V	R
Ejemplo manual	A	R	R	V
Documento	R	A	A	A

Tabla 24. Matriz de contribuciones.

Cada integrante mantiene una bitácora individual en docs/bitacora/, con una fila por sesión: fecha, conocimiento trabajado, actividad, fuente, lo aprendido, dónde se aplicó (archivo o commit), dificultad y solución, y horas.

Anexo C. Resultados por ruta

ρ = saturación de la flota; NA = vueltas no atendidas; $L_{\max}$ en horas. Datos completos en `results/E1_37_rutas.csv` y en la hoja *Resultados por ruta* del Excel.

Ruta	Buses	Vueltas	ρ	NA T0	LS NA T1	LS $L_{\max}$ T1	LPT NA T1	LPT $L_{\max}$ T1	LS NA T2	LPT NA T2
RTI-01	35	103	0.52	0	0	7.77	3	9.73	4	13
RTI-02	27	127	0.87	15	19	10.74	36	10.70	29	43
RTI-04	42	144	0.55	1	2	8.64	4	9.04	5	16
RTI-05	44	150	0.48	0	1	7.55	0	7.87	2	3
RTI-06	32	144	0.56	0	3	8.53	0	8.67	4	8
RTI-07	39	144	0.64	4	8	9.46	15	9.90	11	28
RTI-08	20	108	0.61	0	1	9.23	1	9.31	3	11
RTI-09	43	135	0.54	0	2	9.21	8	9.54	6	17
RTI-10	37	166	0.62	1	4	9.48	11	9.73	7	19
RTU-01	32	144	0.79	12	15	11.93	28	10.12	22	39
RTU-02	29	117	0.63	2	4	8.91	12	8.95	8	18
RTU-04	28	144	0.75	5	7	10.53	21	10.35	13	29
RTU-05	24	144	0.82	7	10	11.50	28	9.91	18	36
RTU-07	28	127	0.46	0	0	7.03	0	7.11	1	0
RTU-08	26	114	0.55	0	2	8.64	2	8.77	3	6
RTU-09	21	120	0.65	0	1	9.70	5	9.60	4	12
RTU-10	16	103	0.61	0	0	9.11	2	9.26	1	6
RTU-11A	21	103	0.42	0	0	6.22	0	6.13	0	0
RTU-11B	22	103	0.40	0	0	6.06	0	6.16	0	0
RTU-12	39	150	0.52	2	3	7.62	0	7.77	4	5
RTU-13	37	144	0.65	3	8	9.32	16	11.65	11	26
RTU-14	37	108	0.42	0	0	6.35	0	6.21	0	0
RTU-15	30	127	0.58	1	4	9.21	3	9.58	5	11
RTU-16	40	150	0.53	2	2	7.86	0	7.97	5	10
RTU-17	28	127	0.57	0	2	8.72	1	8.95	3	7
RTU-18	35	166	0.58	0	4	8.54	0	10.12	5	8
RTU-19	34	144	0.70	6	10	9.51	21	9.44	14	31
RTU-20	31	154	0.55	0	2	7.87	2	7.86	4	6
RTU-21	26	166	0.59	0	0	8.86	0	9.00	0	6
RTU-22	19	127	1.02	21	25	12.94	39	12.83	37	52
RTU-23	37	166	0.60	1	5	9.23	4	9.39	5	19
RTU-24	35	117	0.58	2	4	9.42	7	9.70	7	22
RTU-25	27	127	0.56	0	2	8.29	1	8.42	3	5

Ruta	Buses	Vueltas	ρ	NA T0	LS NA T1	LS L _{max} T1	LPT NA T1	LPT L _{max} T1	LS NA T2	LPT NA T2
RTU-26	33	154	0.66	2	5	9.79	16	10.00	9	23
RTU-27	26	166	0.65	0	0	9.73	7	9.84	2	12
RTU-28	31	145	0.69	3	6	10.30	18	10.46	11	28
RTU-29	30	139	0.72	5	6	10.87	20	11.15	14	28
Total	1141	5017		95	167		331		280	603

Tabla 25. Resultados por ruta (resaltadas: rutas con $\rho \geq 0.85$).